

Bio-ML Agent v0.2 için Antigravity Tabanlı Ultra-Ajan Tasarımı ve Uygulama Yol Haritası

Yönetici Özeti

Bu rapor, https://github.com/zedraxa/bio-ml-agent_v0.2 deposunu (repo) **ultra detaylı** denetleyip; hataları/eksikleri/riskleri çıkarır ve **Antigravity + Claude Opus 4.6** ana runtime varsayımıyla, “her işi yapan” (çok yetkin, dayanıklı, denetlenebilir, güvenli) bir ajan mimarisine **kademeli geçiş** için uygulanabilir bir yol haritası sunar. Mevcut sistem, tek bir çekirdek ajan + “tool-first” politikası + RQ/Redis kuyruk + Gradio arayüz + (opsiyonel) WhatsApp köprüsü ve RAG (Chroma + BM25) gibi bileşenler içerir. Repo’nun README’si, ajanı “otonom hesap/API yönetimi” (Mail.tm ile e-posta açma, web formlarını doldurma, doğrulama kodu girme vb.) gibi riskli bir alana da genişletmeyi hedeflediğini açıkça ifade eder. ¹

Bu hedefe ulaşmak için raporun temel önerileri:

- **Güvenlik ve uyum:** Mevcut “shell=True” BASH çalıştırma (denylist ile) ve dinamik plugin import’u gibi yüksek riskli yüzeyler, enterprise-grade sandbox + allowlist + imza + izolasyon ile yeniden tasarlanmalı. ²
- **Orkestrasyon:** Mevcut **Redis/RQ** tabanlı asenkron işler (worker, job meta progress) korunarak, **Temporal** ile “durable execution” katmanına kademeli geçiş yapılmalı (retry, signal, cancel, event history, replayability). ³
- **Ajan kontrol yüzeyi:** Antigravity’nin “Artifacts + yorumla-iterate” modelini hedef sistemin “denetlenebilirlik standardı” yapın: her kritik adım artifact üretmeli, yorumla düzeltme akışı native olmalı. ⁴
- **Model yönlendirme:** Opus 4.6 ana model; fallback havuzu; **LiteLLM gateway** ile rota/anahtar/limit/bütçe yönetimi + request tag’leme + cost tracking. ⁵
- **Bellek/RAG üretimleştirme:** Mevcut Chroma + BM25 hibrit arama yaklaşımı, provenance/TTL/dedup/kalite metrikleri eklenerek üretime alınmalı veya Qdrant + Mem0/Haystack ile yeniden platformlanmalı. ⁶
- **Browser/vision/click/type:** DOM-öncelikli (Playwright/Browser-Use benzeri) + vision fallback; screenshot/DOM-diff doğrulama; gerçek profil/session yönetimi; 2FA’da “insan müdahalesi” kapısı; kayıtların artifact olarak saklanması. ⁷
- **Çok-kanallı entegrasyon** (OpenClaw benzeri): “tek trust boundary = tek kullanıcı” izolasyonu, yorumlanabilir eylem logları, kill-switch ve görev durdur/iptal sinyalleriyle tasarlanmalı. ⁸

Bu rapor ayrıca: (i) repo dosya-dosya bulgu tablosu, (ii) öncelikli risk listesi (security/privacy/safety/legal), (iii) hedef mimari + mermaid diyagramları, (iv) LangGraph + Temporal orkestrasyon planı, (v) model routing + bellek/RAG + browser pipeline tasarımı, (vi) üretim hardening + observability + CI/CD, (vii) faz/sprint bazlı uygulama planı ve (viii) Antigravity’e kopyala-yapıştır “görev paketleri” içerir.

Kapsam, Varsayımlar ve Hedefler

Kapsam

- **Mevcut repo denetimi:** Repo kök dizinindeki dosyalar ve ana modüller; RQ/Redis kuyruğu; Gradio UI; RAG; WhatsApp client köprüsü; plugin sistemi; config/vault/email bileşenleri. ⁹
- **Antigravity tabanlı ultra-ajan:** Denetlenebilir artifact üretimi; browser recordings; yorumla-iterasyon; agent manager tarzı görev kontrol yüzeyi. ¹⁰
- **Orkestrasyon:** LangGraph (interrupt/HITL) + Temporal (durable execution) entegre planı. ¹¹
- **Model routing:** Opus 4.6 + fallback, Anthropic data residency opsiyonları, LiteLLM gateway. ¹²
- **Güvenlik/uyum:** KVKK/GDPR; WhatsApp ToS/policy; container hardening; secret scanning. ¹³

Varsayımlar (Belirsizler Açıkça İşaretlendi)

- **Infra sağlayıcı: Belirsiz** (AWS/GCP/Azure/on-prem olabilir). Bu rapor, Docker + Kubernetes uyumlu bir tasarım önerir. ¹⁴
- **Bütçe / token limiti: Belirsiz.** Bu nedenle routing + bütçe yönetimi, virtual key, tag bazında spend tracking önerilir. ¹⁵
- **Antigravity erişimi:** Kullanıcıda Antigravity kurulu ve agent manager + artifacts kullanımı mümkün varsayılır. ¹⁶
- **Opus 4.6 kimlik bilgileri:** Kullanıcı sağlayacak varsayılır; ayrıca veri yerleşimi (inference_geo) opsiyonları gereksinime göre seçilecek. ¹⁷

Başarı Kriterleri

- **Denetlenebilirlik:** Her yüksek etkili eylem (dosya yazma, shell, web formu, mesaj gönderme) “kanıt artifact” üretmeli (diff, test raporu, screenshot, recording). ¹⁸
- **Dayanıklılık:** Uzun görevler process ölse bile kaldığı yerden devam edebilmeli (Temporal event history + replay). ¹⁹
- **Güvenlik:** Sandbox/allowlist; secrets sızıntısı önlenmiş; plugin izolasyonu; ağ egress politikaları; kill-switch. ²⁰
- **Uyum:** Sağlık verisi gibi özel nitelikli veri senaryolarında KVKK/GDPR ilkeleriyle teknik kontroller. ²¹

Mevcut Repo Denetimi

Repo Dosya Ağacı ve Yüksek Seviye Bulgular

Aşağıdaki tablo repo kökündeki ana dosya/dizinleri ve **yüksek seviye** bulguları özetler (liste GitHub file tree üzerinden). ⁹

Not: Bu tablo “dosya adı → rol → kritik bulgu/eksik” perspektifiyle hazırlanmıştır. Bazı dosyaların içeriği tek tek açılıp tamamen incelenmediği için “belirsiz” etiketi kullanıldı; buna rağmen dosyanın varlığı repo file tree’den doğruludur. ²²

Dosya/Dizin	Rol	Bulgular / Kritik Eksikler
<code>.github/workflows/</code>	CI iş akışları	Var ama içerik detayları bu raporda doğrulanmadı; secret scanning, test, lint pipeline’ları muhtemelen eksik/dağınık (belirsiz). ²³

Dosya/Dizin	Rol	Bulgular / Kritik Eksikler
<code>Dockerfile</code>	Container build	Mevcut; non-root, multi-stage, minimal image, seccomp/apparmor gibi hardening görünmüyor (ileride eklenmeli). ²⁴
<code>docker-compose.yml</code>	Local/stack ayağa kaldırma	Redis + API + worker + Gradio + MLflow birlikte; <code>.env</code> dosyası container içine mount ediliyor → secrets sızıntısı riski. ²⁵
<code>pyproject.toml</code>	Bağımlılıklar	Core'da <code>chromadb</code> , <code>sentence-transformers</code> , <code>redis</code> , <code>rq</code> ; opsiyonelde <code>fastapi</code> , <code>flask</code> , <code>gradio</code> , <code>openai</code> , <code>anthropic</code> , <code>google-genai</code> var → çoklu backend hedeflenmiş. ²⁶
<code>agent.py</code>	Çekirdek ajan	<code>subprocess.run(... shell=True ...)</code> ile BASH çalıştırma; denylist regex var ama sınırlı; tam sandbox yok. ²⁷
<code>services/agent_service.py</code>	Agent runtime/stream	"Tool-first policy", plan.md yazdırma zorlaması, approval mode (tam otomatik / adım / akıllı / checkpoint) içeriyor → iyi; fakat güvenli tool izinleri granular değil. ²⁸
<code>web_ui.py</code>	Gradio UI	Model seçimi listesi var; onay modu UI'dan kontrol ediliyor; multi-step stream eventleri gösteriyor. ²⁹
<code>job_worker.py</code>	RQ worker	Redis/RQ kuyruk (<code>agent_tasks</code>); job.meta progress güncellemesi; swarm job ve RAG index job var. ³⁰
<code>rag_engine.py</code>	RAG	Chroma yoksa "vektör arama yapamayacaktır" uyarısı; BM25 corpus diskten; pdf/docx/pptx/xlsx parse var. ³¹
<code>plugin_manager.py</code>	Plugin sistemi	<code>importlib.util.spec_from_file_location</code> + <code>exec_module</code> ile dinamik kod yükleme → imzasız/izolasyonsuz RCE yüzeyi. ³²
<code>utils/config.py</code>	Pydantic config	Deny patterns regex listesi var; ayrıca config içinde AgentSection ve AgentConfig gibi ikili yapı dikkat çekiyor (tutarlılık borcu). ³³
<code>utils/email_client.py</code>	Otonom email	<code>api.mail.tm</code> ile otomatik hesap açma/token alma; README'de "otonom hesap/API yönetimi" hedeflerinin temel parçası. Abuse/ToS riski yüksek. ³⁴
<code>utils/vault.py</code>	"Kasa"	README "şifreli" dese de (iddia), içerik incelenmeden şifreleme doğrulanamaz; vault tasarımı KMS/encryption gerektirir (belirsiz). ²²

Dosya/Dizin	Rol	Bulgular / Kritik Eksikler
<code>whatsapp-client/</code>	WhatsApp köprüsü	Puppeteer <code>--no-sandbox</code> ; LocalAuth; Node tarafında <code>/push-message</code> API; Python sunucusunu bash ile spawn ediyor. ToS ve güvenlik riski çok yüksek. ³⁵
<code>api_server.py</code>	Backend API	Mevcut (içerik tam doğrulanmadı); compose "FastAPI backend" diyor ama komut <code>python3 api_server.py</code> (framework belirsiz). ³⁶
<code>api_server.log</code> , <code>job_worker.log</code> , <code>test_output.log</code>	Loglar	Repo'ya log dosyaları commit'lenmiş görünüyor → secrets/PHI sızıntısı ve KVKK/GDPR riski. ²²
<code>tests/</code> , <code>test_api.py</code> , <code>test_xai.py</code>	Testler	Test dosyaları var; kapsam/kalite ve e2e replay testleri belirsiz. ³⁷
<code>memory_manager.py</code>	Bellek	Dosya mevcut ancak içerik burada doğrulanmadı; AgentService "memory.store_interaction" çağırıyor. ³⁸
<code>llm_backend.py</code>	Model backend	Dosya mevcut; pyproject'te OpenAI/Anthropic/Google backend opsiyonları var; routing/policy/telemetry yok (belirsiz). ³⁹
<code>GELISTIRME_PLANI.md</code> , <code>V6_YOL_HARITASI.md</code> , <code>RAPOR.md</code> , <code>KULLANMA_KILAVUZU.md</code>	Dokümantasyon	Var; bu rapor Antigravity-style "artifact-first" dokümana dönüştürmeyi önerir. ⁴⁰

Kritik Teknik Gözlemler

1) **Tool-first enforcement güçlü ama "güvenli tool sınırları" zayıf:** AgentService, aksiyon taleplerinde tool kullanmayı zorunlu kılıyor (plan.md yazdırma, BASH/PYTHON/WEB_SEARCH vs.). Bu, agent'in "lafla değil eylemle" ilerlemesini sağlıyor; ancak "hangi tool hangi yetkiyle" konusu daha granular politika ister. ⁴¹

2) **BASH yürütme denylist'i var ama shell=True ve sandbox yok:** agent.py denylist regex ile tehlikeli komutları engellemeye çalışıyor; fakat `shell=True` ile çalıştırma ve denylist'in sınırlı kapsamı, üretimde yeterli değildir. Güvenli çözüm "kural + sandbox + izinli capability" üçlüsüdür. ²⁷

3) **Plugin sistemi direkt RCE yüzeyi:** plugin_manager, klasördeki `.py` dosyalarını importlib ile yükleyip `exec_module` çalıştırıyor; imza/doğrulama/izolasyon yok. Bu, supply-chain saldırısı ve yerel RCE açısından yüksek risk. ³²

4) **WhatsApp köprüsü + otomasyon ToS ve güvenlik açısından kırılgan:** Puppeteer sandbox devre dışı; WhatsApp Business Terms "scrape/extract" ve izinsiz etkileşimleri yasaklar; ayrıca log/kimlik bilgisi sızıntısı riski yüksektir. ⁴²

5) **RAGEngine üretimleştirme seviyesinde değil:** Dosya tipleri ve BM25 + (opsiyonel) vektör arama var; ancak ingestion/provenance/dedup/TTL/kalite ölçümü, incremental indexing stratejisi ve güvenli içerik filtreleme eklenmeli. ⁴³

6) **.env mount ve repo'ya log commit'i**: docker-compose `.env`'yi container içine mount ediyor; repo'da log dosyalarının bulunması, secret scanning ve güvenlik hijyeni açısından kırmızı bayrak. ⁴⁴

7) **README'de "otonom hesap/API edinimi" iddiası, uyum ve güvenlikte özel tasarım gerektirir**: README, ajanın web sitelerine kayıt olup mail doğrulaması yaparak API anahtarı alabileceğini söylüyor. Bu yaklaşım; ToS ihlali, abuse, kimlik bilgisi güvenliği ve KVKK/GDPR açısından en riskli alanlardan biridir. ⁴⁵

Öncelikli Riskler ve Uyum

Aşağıdaki risk listesi **etki × olasılık** mantığıyla önceliklendirildi ve her risk için kötüye kullanım senaryosu + somut mitigasyon verildi.

Öncelikli Risk Tablosu

Öncelik	Risk	Etki	Kötüye kullanım senaryosu	Somut mitigasyonlar
P0	Yetkisiz kod yürütme (BASH/PYTHON)	Host compromise / veri sızıntısı	Prompt injection ile <code>curl</code> / <code>ssh</code> / <code>exfil</code> komutu; denylist bypass	Zorunlu sandbox (container/VM), egress allowlist, capability bazlı komut API'si, dosya sistemi izinleri, "approve gate". ⁴⁶
P0	PluginManager RCE	Tam RCE + supply-chain	Kötücül plugin dosyası; <code>exec_module</code> ile çalışır	Plugin manifest + imza doğrulama + allowlist; ayrı process/UID; network kapalı; read-only FS; audit log. ⁴⁷
P0	WhatsApp otomasyonu ToS/policy ihlali	Hesap ban, hukuki/kritik operasyon riski	Bulk/izinsiz mesaj; scraping → ban	WhatsApp Business resmi kanalları; ToS uyumlu opt-in; otomasyon sınırları; riskli mod varsayılan kapalı. ⁴⁸
P0	KVKK/GDPR: sağlık verisi (özel nitelikli veri) işleme	İdari yaptırım + itibar kaybı	Klinik JSON, loglarda PHI; veri üçüncü tarafa aktarım	Veri minimizasyonu; maskeleyme; şifreleme; role-based erişim; "yeterli önlemler" seti; ayrı tenancy; audit trail. ⁴⁹
P1	Secrets sızıntısı (log + .env mount)	API anahtarları ele geçirilir	repo'da log dosyası; container'da <code>.env</code> okunur	<code>.env</code> mount kaldır; secrets manager; GitHub secret scanning + pre-commit detect-secrets; redaction. ⁵⁰

Öncelik	Risk	Etki	Kötüye kullanım senaryosu	Somut mitigasyonlar
P1	Browser otomasyonu ile kimlik bilgisi/2FA riski	Hesap ele geçirme	Cookie/profile çalınması; session replay	Profile izolasyonu; 2FA insan kapısı; vault/KMS; kayıtlar artifact; phishing domain allowlist. ⁵¹
P1	Çoklu kullanıcı/çoklu kanal trust boundary	Yetki karışması	Bir kullanıcının aracı diğer kullanıcı adına eylem yapar	OpenClaw modelindeki gibi “one trust boundary per gateway”; tenant isolation; per-user credential store. ⁵²
P2	Observability’de yüksek kardinalite (PII label)	İzleme sistemi çökmesi + PII log	metrics label içine session_id/email koymak	Prometheus naming + low-cardinality label; PII redaction; trace sampling. ⁵³

KVKK/GDPR ve Sağlık Verisi (PHI) İçin Uyum Notları

- KVKK karar özetlerinde “sağlık verisi”nin Kanun’un 6. maddesi kapsamına girdiği ve şartlara dayanmadan aktarımın yaptırma konu olabildiği örnekleniyor; dolayısıyla “klinik JSON” gibi payload’ların işlenmesi ayrı politika ister. ⁵⁴
- KVKK, özel nitelikli kişisel verilerin işlenmesinde ayrıca “Kurul tarafından belirlenen yeterli önlemler” şartına işaret eder; teknik ve idari önlemler check-list’i üretim tasarımına gömülmelidir. ⁵⁵
- GDPR’de “special categories of personal data” (Article 9) sağlık verisi dahil sıkı sınırlamalar getirir; bu nedenle retention/TTL, erişim kontrolü, şifreleme ve veri minimizasyonu “default” olmalıdır. ⁵⁶

ToS/Policy Uyum: WhatsApp Otomasyonu

Repo, WhatsApp Web tabanlı bir köprü içeriyor. Buna karşılık WhatsApp Business Terms; veri kazıma/scrape, izinsiz uygulama etkileşimi gibi faaliyetleri yasakladığını açıkça belirtir. Bu yüzden “ultra ajan” tasarımında WhatsApp entegrasyonu **resmi API/policy uyumlu** bir mod olarak ele alınmalı, “web scraping/automation” default kapalı olmalıdır. ⁵⁷

Hedef Mimari

Bu bölüm, Bio-ML Agent’ı “ultra ajan” seviyesine taşıyacak **modüler mimariyi**, veri akışlarını, arayüzleri ve güvenlik sınırlarını tanımlar. Antigravity’nin “Artifacts” yaklaşımı, burada “kanıta dayalı ajan” standardı olarak alınmıştır. ⁵⁸

Hedef Modüller ve Arayüzler

Ajanı “her işi yapan” hale getirme iddiasını teknik olarak sürdürülebilir kılmak için ajanı tek bir monolit yerine şu katmanlara böleriz:

1) Control Plane (Kontrol Düzlemi)

- Görev ajandası, görev durdur/iptal, müdahale, yorum, onay kapıları, kill-switch
- Temporal workflow state + LangGraph interrupt + Antigravity artifact/yorum uyumu ⁵⁹

2) Execution Plane (Yürütme Düzlemi)

- Sandbox runtime (container/VM), browser runtime, dosya sistemi, network egress - OpenHands runtime yaklaşımından ilham: "runtime = bash/browser/fs/env/plugin" soyutlaması ⁶⁰

3) Intelligence Plane (Zeka Düzlemi)

- Model routing (Opus 4.6 ana), tool-selection, planner, verifier (hakimiyet/kanıt)
- LiteLLM gateway ile policy/cost yönetimi ⁶¹

4) Knowledge Plane (Bilgi Düzlemi)

- Çok katmanlı bellek (episodic/semantic/procedural/preferences/policy)
- RAG ingestion + provenance + incremental indexing + quality metrics
- Qdrant payload indexing / filtering + Mem0 Graph Memory + Haystack hybrid retrieval ⁶²

5) Connector Plane (Bağlayıcılar / Kanallar)

- Email, Slack, GitHub, WhatsApp (policy uyumlu), webhooks
- OpenClaw benzeri "tek trust boundary" izolasyonu kritik ⁵²

Mimari Diyagram

```
flowchart TB
    subgraph CP[Control Plane]
        UI[Antigravity/Gradio UI + Task Ajandası]
        APPR[Approval Gates & Kill-Switch]
        ART[Artifact Store (plans, diffs, screenshots, recordings)]
    end

    subgraph ORCH[Orchestration]
        LG[LangGraph Graph (interrupt/HITL)]
        TMP[Temporal Workflows (durable execution)]
    end

    subgraph IP[Intelligence Plane]
        ROUTER[Model Router (Opus 4.6 primary)]
        VER[Verifier Layer (tests/schema/evidence)]
        POL[Policy Engine (tool allowlist, risk scoring)]
    end

    subgraph EP[Execution Plane]
        SB[Sandbox Runtime (container/VM)]
        BROW[Browser Runtime (DOM-first + vision fallback)]
        TOOLS[Tool Adapters (fs, bash, git, http)]
    end

    subgraph KP[Knowledge Plane]
        MEM[Multi-layer Memory (episodic/semantic/procedural)]
        RAG[RAG Pipeline (ingest, provenance, index)]
```

```

    VDB[Vector DB (Qdrant/Chroma)]
end

subgraph CONN[Connector Plane]
    GATE[Gateway (OpenClaw-style trust boundary)]
    CH[Channels: email/chat/webhooks]
end

UI --> LG
UI --> TMP
LG --> ROUTER
TMP --> ROUTER
ROUTER --> POL
POL --> TOOLS
TOOLS --> SB
TOOLS --> BROW
SB --> VER
BROW --> VER
VER --> ART
ROUTER --> MEM
RAG --> VDB
MEM --> RAG
GATE --> CH
CH --> UI
ART --> UI

```

Bu diyagramın felsefesi: **“Ajan = eylem + kanıt + denetim”**. Antigravity’nin artifacts (plan, diff, screenshot, recording) yaklaşımı da tam olarak “log yerine denetlenebilir deliverable” üretmeyi hedefler.

63

Veri Akışları ve Kanıt Zinciri

Kanıt zinciri (evidence chain) üretimde kritik bir güvenlik özelliği olarak ele alınmalı:

- Her tool çağırısı: (i) talep, (ii) policy kararı, (iii) sandbox sonucu, (iv) verifier doğrulaması, (v) artifact üretimi ile kaydedilir. Antigravity dokümanları artifacts, screenshot ve browser recording gibi somut çıktıları bu amaçla tanımlar. ⁶⁴
- LangGraph interrupt/HITL: kritik tool call öncesi duraklatıp onay/editleme/reject sağlanabilir; state persistence ile kaldığı yerden devam eder. ⁶⁵
- Temporal: uzun işlerde event history üzerinden recovery/replay sağlar. ¹⁹

Orkestrasyon ve Durable Execution

Repo bugün **Redis/RQ** ile job kuyruğu kullanıyor (agent_tasks) ve job meta ile progress yazıyor. ⁶⁶
Hedef: Bunu kırmadan, **Temporal** ile “durable execution”a kademeli geçmek.

Orkestrasyon Seçenekleri Karşılaştırma Tablosu

Seçenek	Güçlü Yan	Zayıf Yan	Repo'ya geçiş stratejisi
Mevcut: RQ/Redis	Basit, hızlı kurulum; mevcut worker	Durable/replay zayıf; uzun işlerde kırılgan	Kısa vadede koru; Temporal'a "bridge" yap
LangGraph (tek başına)	Agent mantığı/graph; interrupt ile HITL	Durable execution değil; dış persistence gerekir	Graph mantığını koru; Temporal workflow içine yerleştir
Temporal (tek başına)	Dayanıklılık, retry/ signal/cancel; event history	Agent "plan/tool" mantığı için ek katman gerekir	Tool call'ları activity'lere böl; agent state'ini workflow'a koy
LangGraph + Temporal (önerilen)	HITL + durable; replay + approval doğal	Doğru sınırlar çizilmezse karmaşıklaşır	"Graph = karar", "Temporal = yürütme & state" olarak katmanla

Temporal'ın retry policy kavramı, activity failure ve workflow execution davranışları, signal/cancel gibi mesajlaşma mekanizmaları; burada "ajanda, görev durdurma, müdahale ve yorum" özelliklerini doğal hale getirir. ⁶⁷

Kademeli Geçiş Planı (Redis/RQ → Temporal)

Aşama 1: "Shadow Temporal"

- RQ job çalışırken aynı "workflow id" ile Temporal tarafında sadece event log tut (dry-run).
- Amaç: iş tiplerini sınıflandırmak, retry/cancel gereksinimlerini ölçmek.

Aşama 2: "Dual-write"

- Kritik olmayan işler (ör. doküman indeksleme) Temporal activity olarak çalışsın. Mevcut RQ işlerinin bir kısmını Temporal'a taşı. RAG indexing job mevcutta var. ⁶⁸

Aşama 3: "Control Plane Temporal"

- Onay, checkpoint, kill-switch, ajanda state'i Temporal workflow'lara taşınır.
- UI/Antigravity yüzeyi workflow status'u gösterir ve signal ile "devam/iptal/düzenle" gönderir. Temporal CLI bile "list/query/cancel" gibi operasyonları destekler. ⁶⁹

Aşama 4: "RQ deprecate"

- RQ sadece legacy job'lar için kalır; yeni işler Temporal zorunlu olur.

Workflow DAG Örneği (Agent Run)

```
flowchart LR
    START([Start]) --> PLAN[Plan Node]
    PLAN -->|needs approval| HITL{Interrupt?}
    HITL -->|approve| TOOL[Run Tool Activity]
    HITL -->|edit| PLAN
    HITL -->|reject| FAIL([Stop + Explain])
    TOOL --> VERIFY[Verifier Activity]
    VERIFY -->|pass| ART[Write Artifact + Update Memory]
```

```
VERIFY -->|fail| PLAN  
ART --> DONE([Done])
```

LangGraph interrupts, graph state'i persistence layer ile saklayıp dış input bekleyebilir; bu, "yorum yaz → ajan devam et" döngüsünü doğal yapar. ⁶⁵

Temporal ise event history ile durable execution sağlar; crash sonrası devam edebilir. ⁷⁰

Durable Execution Detayları (Temporal)

- **Retry:** Activity retry policy ile transient hatalar (network, rate limit) otomatik yönetilir. ⁷¹
- **Signal/Cancel:** Ajandadan "dur", "devam et", "bu adımı atla", "bu parametreyi değiştir" gibi müdahaleler signal; "kill-switch" cancel ile yapılır. ⁷²
- **Replayability:** Event history sayesinde workflow deterministik ise replay edilebilir; agent-run replay testleri bu mantıkla tasarlanır. ⁷⁰

Model Routing, Bellek/RAG ve Browser/Vision

Bu bölüm "ultra agent"ın üç temel motorunu (model routing, memory/RAG, browser/vision) birlikte üretmiş şekilde tanımlar.

Model Routing Tasarımı (Opus 4.6 Ana + Fallback + LiteLLM)

Neden gateway?

- LiteLLM Proxy; çoklu sağlayıcıyı tek API yüzeyi altında toplayıp authz, cost tracking, multi-tenant yönetim ve virtual key gibi özellikler sunar. ⁷³
- Virtual Keys ile "hangi takım/hangi ajan hangi modeli kullanır" ve "bütçe limiti" kontrol edilebilir. ⁷⁴
- Cost tracking dokümantasyonu request tags ve kullanıcı ajanı üzerinden etiketlemeyi vurgular. ⁷⁵

Opus 4.6 ile ilgili operasyonel notlar

- Claude 4.6 için data residency `inference_geo` parametresi mevcut; US-only inference fiyat çarpanı uygular; rate limits `inference_geo` değerleri arasında paylaşılabılır. ¹⁷
- Anthropic Privacy Center, API verisi için varsayılan 30 gün backend silme ve API datanın training'de kullanılmaması (aksi anlaşma yoksa) gibi prensipleri açıklar; bu, enterprise uyum tasarımında önemlidir. ⁷⁶

Routing politikası (öneri)

- **Primary:** Claude Opus 4.6 → kompleks planlama, büyük refactor, çok adımlı browser işleri. ⁷⁷
- **Secondary:** Claude Sonnet 4.6 / daha ucuz modeller → rutin görevler, kısa QA, küçük diffs. (Model seçim stratejisi "capability/speed/cost" gibi kriterlerle yapılmalı.) ⁷⁸
- **Local fallback:** Qwen/Llama gibi yerel modeller → hassas veri (PHI) varsa veya offline/airgap mod. (Repo zaten Qwen model isimleri kullanıyor.) ⁷⁹

Model routing stratejileri tablosu

Strateji	Ne zaman	Avantaj	Risk	Kontrol
Yetenek-öncelikli	Büyük/karmaşık işler	Kalite	Maliyet	LiteLLM bütçe + tag ⁸⁰
Maliyet-öncelikli	Çok sayıda küçük iş	Ucuz	Kalite düşüşü	Fallback eşikleri
Veri-öncelikli	PHI/KVKK senaryosu	Veri kontrolü	Yetenek kısıtı	Offline routing + redaction ⁸¹

Çok Katmanlı Bellek Tasarımı

Repo'da `memory_manager.py` dosyası var ve AgentService etkileşim depolamaya çalışıyor. ⁸²
Hedef: Belleği "her şeyi her zaman hatırlayan" değil; **katmanlı, TTL'li, politikaya uygun** hale getirmek.

Bellek katmanları

- 1) **Episodic memory**: "Bu işte ne yaptık?" (run-level)
- 2) **Semantic memory**: "Bu repo'da X nerede / hangi kararlar var?"
- 3) **Procedural memory**: "Bir işi nasıl yapıyoruz?" (playbook)
- 4) **Preferences**: Kullanıcı/organizasyon tercihleri (kod stili, limitler)
- 5) **Policy memory**: Yasaklar, egress allowlist, red flags

Mem0'nun Graph Memory yaklaşımı; ilişki/olay bağlamını vektör aramayla birlikte saklamayı tanımlar. ⁸³

DB/Framework karşılaştırması (Qdrant – Mem0 – Haystack)

Bileşen	Tür	Güçlü Yan	Ne için kullanırsınız
Qdrant	Vector DB	Payload indexing + filtering; metadata filtre performansı; üretim opsları ⁸⁴	Semantic memory + RAG index
Mem0	Memory layer	Graph memory; metadata filtering; memory davranışını üretime uygunlaştırma ⁸⁵	Episodic/preferences/policy extraction + retrieval
Haystack	RAG framework	Hybrid retrieval (dense + keyword) üretim pipeline'i; router bileşenleri ⁸⁶	RAG ingestion/retrieval/ generation orkestrasyonu

Retention/TTL önerisi (KVKK/GDPR uyumlu varsayılan)

- Episodic: 7–30 gün (debug/replay için), PHI varsa 7 gün ve şifreli.
- Semantic: repo dokümanları ve koddan türetilen bilgi (PII'siz) uzun süreli.
- Preferences: kullanıcı/tenant kapatınca silinebilir.
- Policy: sürümlemeli ama değişiklik audit'li.

KVKK'de özel nitelikli veri için "yeterli önlemler" gerekliliği ve GDPR'de special category vurgusu nedeniyle, **PHI/sağlık verisi** için bellek default-off veya heavy redaction tavsiye edilir. ⁸⁷

Retrieval/RAG Productionization

Mevcut RAGEngine: Chroma yoksa vektör arama devre dışı; BM25 corpus diskten yükleniyor; pdf/docx/pptx/xlsx/html ayrıştırılmaları var. ³¹

Üretimleşme için eklenmesi gerekenler:

- **Ingestion:** kaynak kimliği (URI/path/commit hash), parse yöntemi, timestamp, lisans/ToS notu
- **Provenance:** her chunk'ın kaynak dosya + sayfa + bölüm metadata'sı (mevcut kısmen var)
- **Incremental indexing:** yalnız değişen dosyaları indexle (git diff tabanlı)
- **Dedup:** chunk fingerprint (simhash/minhash)
- **Kalite metrikleri:** retrieval@k, groundedness, tok/cevap, stale rate
- **Hybrid retrieval:** Repo'da BM25 + vektör birleşimi var; Haystack hibrit yaklaşımı bu motivasyonu destekler. ⁸⁸

Browser/Vision/Click/Type Pipeline

Antigravity standardı: browser actions → screenshot/recording artifacts

Antigravity, browser subagent'in screenshot ve browser recordings üretebildiğini; bunların artifact olarak saklandığını açıkça dokümente eder. ⁸⁹

DOM-first + vision-fallback tasarımı

- **DOM-first:** Playwright benzeri DOM locator'lar; form doldurma; click; typed input; network idle bekleme
- **Vision-fallback:** DOM kilitlenirse veya Canvas/Shadow DOM ağırlıktaysa screenshot üzerinden UI element tespiti ve "görsel doğrulama"
- **Doğrulama:** eylem sonrası screenshot + DOM diff; "beklenen state" matcher; artifact'e otomatik ekleme

Session / real profile yönetimi

Browser Use dokümanları, cloud profile ile cookie/auth state'in kalıcı yönetimini ve 2FA için profile sync yaklaşımını anlatır. ⁹⁰

Güvenlik notu: Profile/cookie elde edilmesi, **insan onayı zorunlu** bir akıştır; ajan tek başına "kimlik bilgisi edinmeye" çalışmamalı (phishing/ToS riski). Browser kaydı artifact olarak saklanmalı. ⁹¹

2FA stratejileri (uyumlu yaklaşım)

- **Yasak:** CAPTCHA/2FA "bypass" (ToS ve güvenlik nedeniyle).
- **İzinli:**
 - Kullanıcı profili (cookie) senkronize eder → ajan aynı session ile çalışır. ⁹²
 - Ajan, 2FA ekranına gelince interrupt + kullanıcıdan onay/manuel giriş ister (Antigravity comments / LangGraph interrupt). ⁹³

Plugin & Sandbox Güvenliği

“Trust boundary” ilkesi (OpenClaw ile uyumlu)

OpenClaw gateway security dokümanları: Bir tool-enabled agent’a birden fazla güvenilmeyen kullanıcı mesaj atıyorsa, hepsinin aynı delegated authority’yi paylaştığını varsayın; desteklenen sınır “1 trust boundary per gateway” yaklaşımıdır. ⁵²

Bu prensibi ultra-ajan tasarımına aynen taşıyın:

- **Tek kullanıcı / tek tenant / tek gateway**
- Tenant başına ayrı credential store, ayrı sandbox, ayrı egress policy

Plugin manifest örneği (skill allowlist + capability)

```
skill:
  name: "bio-data-ingest"
  version: "0.1.0"
  entrypoint: "skills/bio_data_ingest/main.py:run"
  signature:
    type: "ed25519"
    public_key_id: "skills-ci-key-01"
  capabilities:
    filesystem:
      read:
        - "workspace/data/**"
      write:
        - "workspace/artifacts/**"
    network:
      egress_allowlist:
        - "ncbi.nlm.nih.gov"
        - "ebi.ac.uk"
      deny_private_ranges: true
    execution:
      bash:
        allowed_commands:
          - "python"
          - "pip"
        max_seconds: 60
  approvals:
    required_for:
      - "network.egress_allowlist:not_matched"
      - "execution.bash"
      - "filesystem.write:workspace/artifacts/**"
```

Bu yaklaşım, repo’daki “plugin_manager exec_module” tasarımını güvenli hale getirmek için temel çerçevedir. ⁹⁴

Verifier Layer

Repo'da test dosyaları var; ayrıca AgentService "approval mode" ve tool-first enforcement içeriyor. ⁹⁵
Bunları "kanıtlayıcı" bir katmana dönüştürmek gerekir:

- **Schema checks:** tool input/output JSON schema; RAG provenance schema
- **Unit tests:** core policy (denylist/capabilities), memory TTL, router
- **Integration tests:** sandbox + tool adapter + LangGraph interrupt
- **E2E:** "agent-run replayability" (Temporal event history replay) ⁷⁰
- **Hallucination audits:** RAG kanıtı yoksa "fail-closed" veya "düşük güven" etiketi; artifact üzerinde "kanıt linkleri" zorunlu

LangGraph HITL middleware, hassas tool'larda interrupt ile approve/edit/reject akışını tanımlar; bu verifier katmanının "insan kapısı" mekanizmasıdır. ⁹⁶

CI/CD, Docker ve Production Hardening

- OWASP Docker Security Cheat Sheet: container güvenlik hataları ve best practice listesi sunar. ⁹⁷
- Docker USER instruction / non-root: root çalışmanın risklerine ve USER kullanımına vurgu yapılır. ⁹⁸
- Docker rootless mode: daemon ve container'ları non-root çalıştırarak risk azaltma. ⁹⁹
- GitHub secret scanning: commit'lenen credentials'ları tespit edip erken müdahale sağlar. ¹⁰⁰
- OpenTelemetry Python: manual instrumentation ile traces/metrics/logs üretimi. ¹⁰¹
- Prometheus label naming: yüksek kardinalite label (user_id/email) kullanılmamalı. ¹⁰²

Uygulama Planı ve Antigravity Görev Paketleri

Bu bölüm "en az 10 sayfa" yoğunluğunda uygulanabilir planı; fazlar/sprintler, efor tahmini, bağımlılıklar, kabul kriterleri, dosya bazlı değişiklik listesi ve Antigravity'e verilecek **kopyala-yapıştır görev paketleri** ile sunar. Antigravity'nin artifacts/yorum mekanizması, tüm fazlarda "çıktı standardı" kabul edilir. ⁴

Fazlar ve Sprintler (Özet Tablo)

Efor tahminleri **kişi-gün** cinsinden yaklaşık verilmiştir (infra/bütçe belirsizliği nedeniyle $\pm$ %30 oynayabilir).

Faz	Sprint	Amaç	Tahmini efor	Bağımlılıklar	Kabul kriteri
Faz A	S1	Repo hijyeni + risk kapatma	6-10	—	Log/secrets temiz; CI temel test çalışır
Faz A	S2	Sandbox + plugin güvenliği v0	10-15	S1	Plugin imzalı/izole; BASH policy enforced
Faz B	S3	Temporal control-plane + RQ bridge	12-18	S2	Cancel/signal/retry; job replay kanıtı
Faz B	S4	LangGraph graph + HITL interrupt	8-14	S3	Onay kapıları; yorumla-devam akışı

Faz	Sprint	Amaç	Tahmini efor	Bağımlılıklar	Kabul kriteri
Faz C	S5	Model routing + LiteLLM gateway	8-12	S2	Budget, tags, fallback çalışır
Faz C	S6	Memory/RAG productionization	12-20	S5	Provenance/dedup/TTL/metrikler
Faz D	S7	Browser/vision pipeline üretim	12-20	S4	DOM-first + vision fallback + kanıt
Faz D	S8	Observability + governance + kill-switch	8-14	S3	OTel/metrics/audit; kill-switch

Dosya Bazlı Değişiklik Listesi (Commit-Ready)

Aşağıdaki liste “ekle/değiştir/sil” olarak tasarlandı. Repo’daki kritik risklere doğrudan dokunan noktalar (BASH, plugins, vault, WhatsApp) önceliklidir. ¹⁰³

Silinecek / repo’dan çıkarılacaklar

- `api_server.log`, `job_worker.log`, `test_output.log` → **repo’dan çıkar**, `.gitignore`’a al, secret scanning tetikle. ¹⁰⁴
- WhatsApp entegrasyonu “default enable” ise: prod hedefte kapatılmalı; yalnız policy uyumlu mod bırakılmalı. ¹⁰⁵

Değiştirilecekler (yüksek öncelik)

- `agent.py`: `run_bash` ve `run_python` sandbox adapter’a taşınacak; “shell=True denylist” yerine capability bazlı komut API’si, container izolasyonu. ¹⁰⁶
- `plugin_manager.py`: imzasız `exec_module` kaldır; plugin manifest + signature verify + ayrı process/UID. ¹⁰⁷
- `utils/email_client.py`: otomatik hesap açma özelliği varsayılan kapalı; sadece “kullanıcı onayı + policy allowlist” ile. ³⁴
- `docker-compose.yml`: `.env` mount kaldır; secrets injection (K8s secret / docker secrets) ve non-root image. ¹⁰⁸

Eklenecek yeni dosyalar (önerilen yapı)

- `ultra_agent/` (yeni python package)
- `ultra_agent/control/` (ajanda, approval, kill-switch)
- `ultra_agent/orchestration/temporal/` (workflow/activity)
- `ultra_agent/orchestration/langgraph/` (graph definitions)
- `ultra_agent/runtime/sandbox/` (container runner)
- `ultra_agent/runtime/browser/` (DOM-first + vision fallback)
- `ultra_agent/security/policy.py` (capabilities, egress)
- `ultra_agent/security/signing.py` (plugin signature)
- `ultra_agent/observability/otel.py` (OTel init)
- `ultra_agent/memory/` (mem schemas, TTL)
- `ultra_agent/rag/` (ingestion, dedup, provenance)
- `configs/litellm.yaml` (gateway config; budgets, tags)

- `.github/workflows/ci.yml` (pytest, lint, secret scan)
- `SECURITY.md` (threat model + disclosure)
- `ARTIFACTS_SPEC.md` (Antigravity-style çıktı standardı)

Kod Skeçleri (Örnek, Kısa)

1) Tool policy arayüzü

```
# ultra_agent/security/policy.py
from dataclasses import dataclass
from typing import Iterable

@dataclass(frozen=True)
class EgressRule:
    allow_hosts: set[str]
    deny_private_ranges: bool = True

@dataclass(frozen=True)
class CapabilityPolicy:
    can_write_paths: set[str]
    can_read_paths: set[str]
    egress: EgressRule
    allow_commands: set[str]
    max_seconds: int

def authorize_tool_call(tool: str, params: dict, policy: CapabilityPolicy) -
> tuple[bool, str]:
    # return (allowed, reason)
    # 1) tool allowlist
    # 2) path enforcement
    # 3) network host allowlist
    # 4) command allowlist
    return True, "ok"
```

Bu yaklaşım, mevcut regex denylist'in ötesine geçer ve OWASP container/sandbox önerileriyle uyumludur. ¹⁰⁹

2) Temporal workflow “kill-switch” (concept)

```
# ultra_agent/orchestration/temporal/workflows.py
# Pseudocode: Temporal Python SDK ile
# - signal: pause/resume/cancel
# - retry policy: tool activities
# - state: current_step, artifacts, approvals
```

Temporal event history ve signal/cancel mekanikleri durable execution temelidir. ¹¹⁰

3) LangGraph interrupt ile HITL kapısı

```
# ultra_agent/orchestration/langgraph/graph.py
# Pseudocode:
# if tool_call is sensitive:
#     interrupt({"tool": tool_name, "payload": payload, "risk": score})
# resume with approve/edit/reject
```

LangGraph interrupts; state'i saklayıp dış input beklemek için tasarlanmıştır. ⁶⁵

Open-Source Entegrasyon Haritası (Repo Linkleri + Ne Alacağız?)

Aşağıdaki tabloda, istenen projelerin **tam GitHub linkleri**, hangi bileşenlerin adapte edileceği ve repo içinde hangi dosyaların etkileneceği verildi.

Proje	GitHub link	Entegre edilecek parça	Bizde değişecek yer
LangGraph	https://github.com/langchain-ai/langgraph	interrupt/HITL desenleri, graph orkestrasyonu ⁶⁵	<code>services/agent_service.py</code> → <code>ultra_agent/orchestration/langgraph/</code>
Temporal	https://github.com/temporalio/sdk-python	workflow/activity durable execution, retry, signal/cancel ¹¹¹	<code>job_worker.py</code> köprüsü → <code>ultra_agent/orchestration/temporal/</code>
LiteLLM	https://github.com/BerriAI/litellm	Proxy gateway, virtual keys, cost tracking/tags ⁷³	<code>llm_backend.py</code> → gateway client; <code>configs/litellm.yaml</code>
Qdrant	https://github.com/qdrant/qdrant	Vector DB + payload indexing/filtering ¹¹²	<code>rag_engine.py</code> / <code>memory_manager.py</code> → Qdrant adapter
Haystack	https://github.com/deepset-ai/haystack	Hybrid retrieval pipeline, routers ⁸⁶	<code>rag_engine.py</code> → <code>ultra_agent/rag/</code> pipeline
Mem0	https://github.com/mem0ai/mem0	Memory layer + graph memory opsiyonları ¹¹³	<code>memory_manager.py</code> → Mem0 integration
Browser Use	https://github.com/browser-use/browser-use	Auth profile/cookie sync, production sandbox ¹¹⁴	<code>browser_agent.py</code> → DOM-first pipeline; <code>ultra_agent/runtime/browser/</code>
Crawl4AI	https://github.com/unclecode/crawl4ai	LLM-friendly crawling, markdown extraction ¹¹⁵	RAG ingestion (web kaynakları)

Proje	GitHub link	Entegre edilecek parça	Bizde değışecek yer
OpenHands	https://github.com/OpenHands/OpenHands ve https://github.com/OpenHands/software-agent-sdk	runtime soyutlaması (bash/browser/fs/git), sandbox/workspace ¹¹⁶	<code>agent.py</code> tool adapter'ları → runtime layer
OpenClaw	https://github.com/openclaw/openclaw	gateway trust boundary, kanal entegrasyon modeli ⁵²	Connector plane: per-tenant gateway + policy

Antigravity Görev Paketleri (Kopyala-Yapıştır)

Aşağıdaki paketler, Antigravity'de agent manager'a "görev" olarak verilecek biçimde yazıldı. Her faz için **en az 8 görev** var. Her görev "çıkış artifact'leri" ve "kabul kriterleri" tanımlar (Antigravity artifacts yaklaşımı). ¹¹⁷

Faz A Görev Paketi (S1-S2): Repo Hijyeni + Risk Kapatma

MISSION: Faz A / S1-1 – Repo Hijyeni: Log & Secret Temizliği
GOAL: Repo'da commit'li log dosyalarını ve olası secrets izlerini temizle, CI'da secret scanning ekle.
ARTIFACTS:
- Artifact: "security_repo_hygiene.md" (bulgular + yapılanlar)
- Artifact: "git_diff_summary.md" (silinen dosyalar, .gitignore değışikleri)
ACCEPTANCE:
- api_server.log / job_worker.log / test_output.log repo'dan çıkarıldı
- Secret scanning CI adımı eklendi

MISSION: Faz A / S1-2 – docker-compose Güvenlik İncelemesi
GOAL: docker-compose.yml içinde .env mount ve açık port risklerini analiz et ve düzeltme önerisi üret.
ARTIFACTS:
- "compose_security_audit.md" (riskler + öneriler)
ACCEPTANCE:
- .env mount kaldırma planı + alternatif secrets yöntemi belirlendi

MISSION: Faz A / S1-3 – BASH/PYTHON Tool Risk Analizi
GOAL: agent.py içerisindeki subprocess kullanımını (shell=True vb.) risklendir, bypass senaryoları yaz, mitigasyon tasarla.
ARTIFACTS:
- "tool_rce_threat_model.md"
ACCEPTANCE:
- En az 10 bypass vektörü + karşı önlem listesi

MISSION: Faz A / S1-4 – Plugin Sistemi Sertleştirme Tasarımı

GOAL: plugin_manager.py exec_module yaklaşımını kaldırarak güvenli plugin mimarisi tasarla (manifest+imza+izolasyon).

ARTIFACTS:

- "plugin_security_design.md"
- "skill_manifest_example.yaml"

ACCEPTANCE:

- İmza doğrulama + allowlist + process isolation net

MISSION: Faz A / S1-5 – WhatsApp Köprüsü ToS/Uyum İncelemesi

GOAL: WhatsApp entegrasyonunun ToS/policy risklerini analiz et; uyumlu yol öner (resmi API, opt-in).

ARTIFACTS:

- "whatsapp_policy_risk.md"

ACCEPTANCE:

- Varsayılan kapatma + uyumlu alternatif net

MISSION: Faz A / S2-1 – Sandbox Runtime v0 Spike

GOAL: OpenHands runtime yaklaşımından ilhamla minimal sandbox runner skeleton oluştur.

ARTIFACTS:

- "runtime_sandbox_spike.md"
- Kod: ultra_agent/runtime/sandbox/*

ACCEPTANCE:

- Basit komut allowlist'i ile container içinde çalıştırma PoC

MISSION: Faz A / S2-2 – Vault/Kimlik Bilgisi Tasarımı

GOAL: Vault'u KMS/encryption + per-tenant isolation ile tasarla; plaintext risklerini kapat.

ARTIFACTS:

- "vault_security_arch.md"

ACCEPTANCE:

- KMS opsiyonu + local dev fallback + key rotation planı

MISSION: Faz A / S2-3 – "Otonom API Edinimi" Güvenlik Politikası

GOAL: Ajanın kendi kendine hesap açması/anahtar edinmesi özelliklerini güvenli sınırlar içine al (default off, insan kapısı, allowlist).

ARTIFACTS:

- "api_provisioning_policy.md"

ACCEPTANCE:

- ToS ihlali risklerini engelleyen policy seti + interrupt akışı

Faz B Görev Paketi (S3-S4): Temporal + LangGraph Orkestrasyon

MISSION: Faz B / S3-1 – Temporal Mimari Tasarım

GOAL: Mevcut RQ/Redis yapısını bozmadan Temporal'e geçiş planı ve workflow şemaları üret.

ARTIFACTS:

- "temporal_migration_plan.md"
- Mermaid: "workflow_dag.md"

ACCEPTANCE:

- Dual-run planı + rollback stratejisi + efor tahmini

MISSION: Faz B / S3-2 – Cancel/Signal/Kill-Switch Spesifikasyonu

GOAL: Ajanda üzerinden durdur/devam/iptal/müdahale sinyallerini tanımla.

ARTIFACTS:

- "killswitch_spec.md"

ACCEPTANCE:

- Signal isimleri, payload şemaları, UI davranışı

MISSION: Faz B / S3-3 – RQ Bridge Uygulaması

GOAL: job_worker.py içindeki job tiplerinden birini Temporal'e taşı (ör. index_documents_job).

ARTIFACTS:

- "rq_to_temporal_bridge.md"
- Kod diff

ACCEPTANCE:

- Temporal activity ile aynı iş başarıyla çalışıyor

MISSION: Faz B / S3-4 – Replayability Test Planı

GOAL: Temporal event history replay mantığına dayalı agent-run replay test tasarla.

ARTIFACTS:

- "replay_test_plan.md"

ACCEPTANCE:

- 5 senaryo + kabul ölçütleri

MISSION: Faz B / S4-1 – LangGraph Graph Topolojisi

GOAL: Plan→Tool→Verify→Artifact döngüsünü LangGraph olarak tasarla; interrupt noktalarını belirle.

ARTIFACTS:

- "langgraph_design.md"
- Mermaid graph

ACCEPTANCE:

- Hassas tool'larda interrupt listesi

MISSION: Faz B / S4-2 – Human-in-the-loop Middleware
GOAL: Tool çağrılarını policy'ye göre approve/edit/reject eden middleware prototipi yaz.
ARTIFACTS:
- Kod: ultra_agent/orchestration/langgraph/hitl.py
ACCEPTANCE:
- En az 3 tool için kapı çalışıyor

MISSION: Faz B / S4-3 – Artifact Standardı (Antigravity Uyumlu)
GOAL: Her adımda üretilen artifact türlerini (plan/diff/test/screenshot) standartlaştır.
ARTIFACTS:
- "ARTIFACTS_SPEC.md"
ACCEPTANCE:
- Artifact isimlendirme + metadata şeması

MISSION: Faz B / S4-4 – Comment-to-Iterate Akışı
GOAL: Artifact yorumunu alıp graph state'e geri besleyen mekanizma tasarla.
ARTIFACTS:
- "comment_feedback_loop.md"
ACCEPTANCE:
- Yorum → interrupt resume → yeni artifact versiyonu

Faz C Görev Paketi (S5-S6): LiteLLM Routing + Memory/RAG

MISSION: Faz C / S5-1 – LiteLLM Gateway Kurulumu
GOAL: Proxy'yi virtual keys + DB ile ayağa kaldır; anahtar/bütçe yönetimini etkinleştir.
ARTIFACTS:
- "litellm_setup.md"
- configs/litellm.yaml
ACCEPTANCE:
- En az 2 model route ediliyor; key bazlı limit var

MISSION: Faz C / S5-2 – Routing Politikası (Opus 4.6 primary)
GOAL: İş türüne göre model seçimi (capability/speed/cost/data) politikasını kodla.
ARTIFACTS:
- "routing_policy.md"
- Kod: ultra_agent/control/router.py
ACCEPTANCE:
- 10 örnek istekte doğru model seçimi

MISSION: Faz C / S5-3 – Cost Tracking + Tags
GOAL: request_tags ile iş/tenant/agent bazlı maliyet raporu çıkar.

ARTIFACTS:

- "cost_tracking_dashboard.md"

ACCEPTANCE:

- Tag bazlı spend raporu örneği

MISSION: Faz C / S5-4 – Anthropic Data Residency Parametreleri

GOAL: inference_geo kullanımına uygun config ve test planı hazırla.

ARTIFACTS:

- "data_residency_plan.md"

ACCEPTANCE:

- US/global routing senaryoları

MISSION: Faz C / S6-1 – Memory Katmanları Şema Tasarımı

GOAL: episodic/semantic/procedural/preferences/policy şemalarını çıkar; TTL belirle.

ARTIFACTS:

- "memory_schema.md"

ACCEPTANCE:

- TTL ve redaction kuralları net

MISSION: Faz C / S6-2 – Qdrant Adapter

GOAL: Qdrant payload indexing/filtering ile semantic memory store prototipi kur.

ARTIFACTS:

- Kod: ultra_agent/memory/qdrant_store.py

ACCEPTANCE:

- Metadata filtreli arama çalışıyor

MISSION: Faz C / S6-3 – RAG Provenance & Dedup

GOAL: RAG chunk'larına provenance ekle; dedup mekanizması uygula.

ARTIFACTS:

- "rag_provenance_dedup.md"

ACCEPTANCE:

- Aynı içerik iki kez indexlenmiyor

MISSION: Faz C / S6-4 – Hybrid Retrieval Kalite Deneyi

GOAL: BM25 + dense hibrit aramanın kalite metriklerini ölç ve raporla.

ARTIFACTS:

- "rag_quality_metrics.md"

ACCEPTANCE:

- retrieval@k + groundedness örnekleri

Faz D Görev Paketi (S7-S8): Browser/Vision + Observability/Governance

MISSION: Faz D / S7-1 – DOM-first Browser Driver
GOAL: Playwright/Browser-Use tabanlı DOM-first action katmanı yaz.
ARTIFACTS:
- Kod: ultra_agent/runtime/browser/dom_driver.py
ACCEPTANCE:
- login gerektirmeyen sitede form doldurma/submit çalışıyor

MISSION: Faz D / S7-2 – Vision Fallback + Screenshot Doğrulama
GOAL: DOM başarısızsa screenshot üzerinden doğrulama/eylem (sadece izinli) katmanını ekle.
ARTIFACTS:
- "vision_fallback_design.md"
ACCEPTANCE:
- Screenshot/DOM-diff artifact üretiliyor

MISSION: Faz D / S7-3 – Session/Profile Yönetimi
GOAL: Profile/cookie yönetimini güvenli hale getir (per-tenant isolation, manual approval).
ARTIFACTS:
- "profile_session_security.md"
ACCEPTANCE:
- 2FA noktalarında interrupt zorunlu

MISSION: Faz D / S7-4 – Browser Recording Artifact Standardı
GOAL: Browser aksiyonlarını recording artifact'e dönüştür.
ARTIFACTS:
- "browser_recording_spec.md"
ACCEPTANCE:
- Her browser run sonrası recording var

MISSION: Faz D / S8-1 – OpenTelemetry Entegrasyonu
GOAL: traces/metrics/logs ile agent run observability ekle.
ARTIFACTS:
- "otel_instrumentation.md"
ACCEPTANCE:
- trace_id loglarda; temel metrikler var

MISSION: Faz D / S8-2 – Prometheus Metric Tasarımı (Low Cardinality)
GOAL: PII içermeyen label tasarımı; cardinality kontrolü.
ARTIFACTS:
- "metrics_naming.md"

ACCEPTANCE:

- user_id/email label yok; örnek dashboard

MISSION: Faz D / S8-3 – Governance ve Audit Trails

GOAL: Tool call audit trail + policy versioning + veri erişim logları.

ARTIFACTS:

- "governance_audit.md"

ACCEPTANCE:

- Her kritik eylem audit event üretiyor

MISSION: Faz D / S8-4 – Human-in-the-loop Approval Gates

GOAL: Kritik tool'larda mandatory approval; manual edit/reject akışı.

ARTIFACTS:

- "approval_gates.md"

ACCEPTANCE:

- En az 5 kritik tool kapısı aktif

“İnsan bağımlılığı minimal olsun” gereksinimi için gerçekçi sınırlar

Repo README'si “ajanın kendi adına mail açıp siteye kayıt olup API anahtarı alması” gibi bir yönü hedefliyor. ¹

Bu, pratikte **yüksek abuse/ToS** riski taşır (özellikle sağlık/klinik veri gibi alanlarda). Bu yüzden önerilen tasarım:

- **Minimal insan:** planlama, rutin kod refactor, test koşma, dokümantasyon, veri temizleme, RAG indexing gibi alanlarda.
- **Zorunlu insan kapısı:** yeni hesap açma, ödeme/abonelik, 2FA, kimlik doğrulama, messaging (WhatsApp/email) gibi eylemlerde. LangGraph interrupt + Antigravity comment akışı bunu sağlar. ¹¹⁸

Son Not: Repo'daki mevcut mimariyi “ultra ajan çekirdeği” olarak konumlama

Mevcut sistemde değerli altyapılar var:

- AgentService'in tool-first zorlaması ve approval mode çeşitleri, modern “agent governance” için iyi bir başlangıç. ¹¹⁹
- RQ worker altyapısı, Temporal'a geçişte “legacy bridge” rolü görebilir. ¹²⁰
- RAGEngine hibrit arama yaklaşımı, daha üretimsel bir RAG sistemine evrilebilir. ⁸⁸

Ama “ultra ajan” hedefi için, güvenlik sınırları (sandbox, plugin, secrets, ToS uyumu) ve dayanıklılık (Temporal) mutlaka güçlendirilmelidir. ¹²¹

¹ ⁹ ²² ²³ ²⁴ ³⁷ ⁴⁰ ⁴⁵ ⁸² ⁹⁵ ¹⁰⁴ https://github.com/zedraxe/bio-ml-agent_v0.2

https://github.com/zedraxe/bio-ml-agent_v0.2

² ²⁷ ⁴⁶ ¹⁰³ ¹⁰⁶ [bio-ml-agent_v0.2/agent.py](https://github.com/zedraxe/bio-ml-agent_v0.2/blob/main/agent.py) at main · zedraxe/bio-ml-agent_v0.2 · GitHub

https://github.com/zedraxe/bio-ml-agent_v0.2/blob/main/agent.py

3 25 36 44 50 108 https://github.com/zedraxe/bio-ml-agent_v0.2/blob/main/docker-compose.yml
https://github.com/zedraxe/bio-ml-agent_v0.2/blob/main/docker-compose.yml

4 10 18 58 63 64 117 <https://antigravity.google/docs/artifacts>
<https://antigravity.google/docs/artifacts>

5 61 73 **LiteLLM - Getting Started**
https://docs.litellm.ai/docs/?utm_source=chatgpt.com

6 31 43 88 **bio-ml-agent_v0.2/rag_engine.py at main · zedraxe/bio-ml-agent_v0.2 · GitHub**
https://github.com/zedraxe/bio-ml-agent_v0.2/blob/main/rag_engine.py

7 90 92 <https://docs.cloud.browser-use.com/guides/authentication>
<https://docs.cloud.browser-use.com/guides/authentication>

8 52 **Security - OpenClaw**
https://docs.openclaw.ai/gateway/security?utm_source=chatgpt.com

11 65 93 **Interrupts - Docs by LangChain**
https://docs.langchain.com/oss/python/langgraph/interrupts?utm_source=chatgpt.com

12 77 <https://docs.anthropic.com/en/docs/about-claude/models>
<https://docs.anthropic.com/en/docs/about-claude/models>

13 21 49 54 81 <https://www.kvkk.gov.tr/Icerik/5364/2018-143>
<https://www.kvkk.gov.tr/Icerik/5364/2018-143>

14 20 97 109 121 https://cheatsheetseries.owasp.org/cheatsheets/Docker_Security_Cheat_Sheet.html
https://cheatsheetseries.owasp.org/cheatsheets/Docker_Security_Cheat_Sheet.html

15 74 **Virtual Keys**
https://docs.litellm.ai/docs/proxy/virtual_keys?utm_source=chatgpt.com

16 <https://codelabs.developers.google.com/getting-started-google-antigravity>
<https://codelabs.developers.google.com/getting-started-google-antigravity>

17 <https://docs.anthropic.com/claude/reference/getting-started-with-the-api>
<https://docs.anthropic.com/claude/reference/getting-started-with-the-api>

19 70 110 111 **Events and Event History | Temporal Platform Documentation**
https://docs.temporal.io/workflow-execution/event?utm_source=chatgpt.com

26 39 https://github.com/zedraxe/bio-ml-agent_v0.2/blob/main/pyproject.toml
https://github.com/zedraxe/bio-ml-agent_v0.2/blob/main/pyproject.toml

28 38 41 119 **bio-ml-agent_v0.2/services/agent_service.py at main · zedraxe/bio-ml-agent_v0.2 · GitHub**
https://github.com/zedraxe/bio-ml-agent_v0.2/blob/main/services/agent_service.py

29 **bio-ml-agent_v0.2/web_ui.py at main · zedraxe/bio-ml-agent_v0.2 · GitHub**
https://github.com/zedraxe/bio-ml-agent_v0.2/blob/main/web_ui.py

30 66 68 120 **bio-ml-agent_v0.2/job_worker.py at main · zedraxe/bio-ml-agent_v0.2 · GitHub**
https://github.com/zedraxe/bio-ml-agent_v0.2/blob/main/job_worker.py

32 47 94 107 **bio-ml-agent_v0.2/plugin_manager.py at main · zedraxe/bio-ml-agent_v0.2 · GitHub**
https://github.com/zedraxe/bio-ml-agent_v0.2/blob/main/plugin_manager.py

33 79 **bio-ml-agent_v0.2/utils/config.py at main · zedraxe/bio-ml-agent_v0.2 · GitHub**
https://github.com/zedraxe/bio-ml-agent_v0.2/blob/main/utils/config.py

34 **bio-ml-agent_v0.2/utils/email_client.py at main · zedraxe/bio-ml-agent_v0.2 · GitHub**
https://github.com/zedraxe/bio-ml-agent_v0.2/blob/main/utils/email_client.py

35 42 [bio-ml-agent_v0.2/whatsapp-client/index.js](https://github.com/zedraxe/bio-ml-agent_v0.2/blob/main/whatsapp-client/index.js) at main · zedraxe/bio-ml-agent_v0.2 · GitHub
https://github.com/zedraxe/bio-ml-agent_v0.2/blob/main/whatsapp-client/index.js

48 57 105 <https://www.whatsapp.com/legal/business-terms>
<https://www.whatsapp.com/legal/business-terms>

51 <https://docs.cloud.browser-use.com/concepts/profile>
<https://docs.cloud.browser-use.com/concepts/profile>

53 102 <https://prometheus.io/docs/practices/naming/>
<https://prometheus.io/docs/practices/naming/>

55 87 <https://www.kvkk.gov.tr/Icerik/4110/2018-10>
<https://www.kvkk.gov.tr/Icerik/4110/2018-10>

56 <https://eur-lex.europa.eu/legal-content/EN/TXT/HTML/?uri=CELEX%3A02016R0679-20160504>
<https://eur-lex.europa.eu/legal-content/EN/TXT/HTML/?uri=CELEX%3A02016R0679-20160504>

59 Handling Signals, Queries, & Updates
https://docs.temporal.io/handling-messages?utm_source=chatgpt.com

60 116 <https://github.com/OpenHands/OpenHands/blob/main/openhands/runtime/README.md>
<https://github.com/OpenHands/OpenHands/blob/main/openhands/runtime/README.md>

62 84 112 <https://qdrant.tech/documentation/concepts/payload/>
<https://qdrant.tech/documentation/concepts/payload/>

67 What is a Temporal Retry Policy?
https://docs.temporal.io/encyclopedia/retry-policies?utm_source=chatgpt.com

69 Temporal CLI workflow command reference
https://docs.temporal.io/cli/workflow?utm_source=chatgpt.com

71 Activity Execution - Temporal Platform Documentation
https://docs.temporal.io/activity-execution?utm_source=chatgpt.com

72 Temporal Workflow Execution Overview
https://docs.temporal.io/workflow-execution?utm_source=chatgpt.com

75 80 Spend Tracking
https://docs.litellm.ai/docs/proxy/cost_tracking?utm_source=chatgpt.com

76 <https://privacy.anthropic.com/en/articles/7996866-how-long-do-you-store-personal-data>
<https://privacy.anthropic.com/en/articles/7996866-how-long-do-you-store-personal-data>

78 <https://docs.anthropic.com/en/docs/about-claude/models/choosing-a-model>
<https://docs.anthropic.com/en/docs/about-claude/models/choosing-a-model>

83 <https://docs.mem0.ai/open-source/features/graph-memory>
<https://docs.mem0.ai/open-source/features/graph-memory>

85 <https://docs.mem0.ai/open-source/overview>
<https://docs.mem0.ai/open-source/overview>

86 https://haystack.deepset.ai/tutorials/33_hybrid_retrieval
https://haystack.deepset.ai/tutorials/33_hybrid_retrieval

89 <https://antigravity.google/docs/screenshots>
<https://antigravity.google/docs/screenshots>

91 96 118 Human-in-the-loop
https://docs.langchain.com/oss/python/langchain/human-in-the-loop?utm_source=chatgpt.com

- 98 <https://www.docker.com/blog/understanding-the-docker-user-instruction/>
<https://www.docker.com/blog/understanding-the-docker-user-instruction/>
- 99 <https://docs.docker.com/engine/security/rootless/>
<https://docs.docker.com/engine/security/rootless/>
- 100 <https://docs.github.com/code-security/secret-scanning/about-secret-scanning>
<https://docs.github.com/code-security/secret-scanning/about-secret-scanning>
- 101 <https://opentelemetry.io/docs/languages/python/instrumentation/>
<https://opentelemetry.io/docs/languages/python/instrumentation/>
- 113 <https://github.com/mem0ai/mem0>
<https://github.com/mem0ai/mem0>
- 114 <https://github.com/browser-use/browser-use>
<https://github.com/browser-use/browser-use>
- 115 <https://docs.crawl4ai.com/core/quickstart/>
<https://docs.crawl4ai.com/core/quickstart/>